



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Rosas Martinez Francisco Javier

N° de Cuenta: 320109766

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 1/Noviembre/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1:

Separar del arco la parte del letrero y las puertas o rejas.

Para resolver el ejercicio, tuve que utilizar blender para poder separar primero el letrero del arco y además separar en dos la reja, a cada modelo le agregue una imagen como textura para mejorar su visualización, por último exporte cada modelo en formato .obj para por último importar cada modelo en mi código.

Bloques de código generado

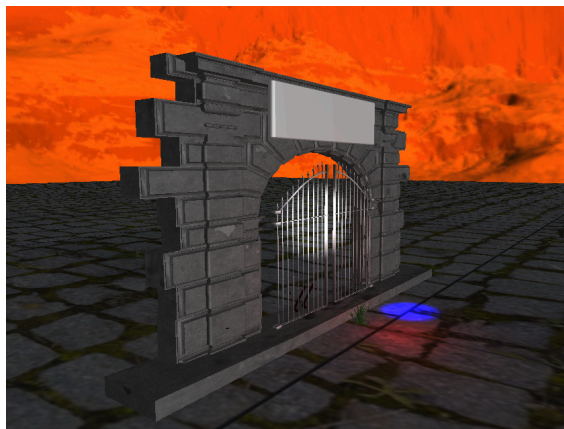
```
//Arco
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -5.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(10.0f, 6.0f, 7.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
arco.RenderModel();

//Letrero
model = modelaux;
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(10.0f, 6.0f, 7.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
letrero.RenderModel();

//Reja 1
model = modelaux;
model = glm::translate(model, glm::vec3(6.0f, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.03f, 0.03f, 0.03f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta.RenderModel();

//Reja 2
model = modelaux;
model = glm::translate(model, glm::vec3(-7, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.03f, 0.03f, 0.03f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta.RenderModel();
```

Código en ejecución



Ejercicio 2:

Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEIHHC desplazándose las letras de derecha a izquierda como si fuera letrero LCD/LED de forma cíclica.

Para resolver el ejercicio, primero hice el mensaje que tendría el letrero usando la fuente de mi universo Brawl Stars. Luego modifiqué la imagen con ayuda de Gimp para que pudiera usarse correctamente. En el código, ajusté los valores u y v del modelo numeroVertices para que la textura se mostrara bien. Por último, le asigné un movimiento a la textura en función de deltaTime, y agregué dos modelos con un pequeño desfase para que se pareciera más a un letrero LED.

Bloques de código generado

```
Model puerta;  
Model arco;  
Model letrero;
```

```
//  
GLfloat numeroVertices[] = {  
//      x      y      z      u      v      nx      ny      nz  
-0.5f, 0.0f, 0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,  
0.5f, 0.0f, 0.5f, 1.0f / 16.0f, 0.0f, 0.0f, -1.0f, 0.0f,  
0.5f, 0.0f, -0.5f, 1.0f / 16.0f, 1.0f, 0.0f, -1.0f, 0.0f,  
-0.5f, 0.0f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f  
};
```

```
letras = Texture("Textures/letrasBS2.png");  
letras.LoadTextureA();
```

```
puerta = Model();  
puerta.LoadModel("Models/puerta4.obj");  
letrero = Model();  
letrero.LoadModel("Models/letrero.obj");  
arco = Model();  
arco.LoadModel("Models/arcop09.obj");
```

```
// letrero 1  
toffsetnumerocambiau += 0.25 * deltaTime * 0.0025;  
if (toffsetnumerocambiau > 1.0)  
    toffsetnumerocambiau = 0.0;  
toffsetnumerov = 0.0;  
toffset = glm::vec2(toffsetnumerocambiau-2.0/16.0, toffsetnumerov);  
model = modelaux;  
model = glm::translate(model, glm::vec3(3.0f, 15.5f, -0.5f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));  
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));  
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(1.0f, 1.0f, 1.0f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
letras.UseTexture();  
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);  
meshList[6]->RenderMesh();  
  
// letrero 2  
toffsetnumerocambiau += 0.25 * deltaTime * 0.0025;  
if (toffsetnumerocambiau > 1.0)  
    toffsetnumerocambiau = 0.0;  
toffsetnumerov = 0.0;  
toffset = glm::vec2(toffsetnumerocambiau, toffsetnumerov);  
model = modelaux;  
model = glm::translate(model, glm::vec3(-3.0f, 15.5f, -0.5f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));  
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));  
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(1.0f, 1.0f, 1.0f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
letras.UseTexture();  
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);  
meshList[6]->RenderMesh();
```

```
// letrero 3
toffsetnumerocambiau += 0.25 * deltaTime * 0.0025;
if (toffsetnumerocambiau > 1.0)
    toffsetnumerocambiau = 0.0;
toffsetnumerov = 0.0;
toffset = glm::vec2(toffsetnumerocambiau-1.0/16.0, toffsetnumerov);
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 15.5f, -0.5f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
letras.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[6]->RenderMesh();
```

Código en ejecución



Ejercicio 3:

Hacer que al presionar una tecla una de las puertas/rejas se abra por medio de una rotación y la otra puerta se abra por medio de un deslizamiento.

Con este ejercicio tuve que agregar el atributo mover a mi clase Window, junto con su método get. Después añadí las teclas M y N, que sirven para activar el movimiento y la traslación de cada puerta. En el main puse los condicionales if para que, cuando mover fuera verdadero, se aumentaran los valores de rotación y traslación hasta llegar a los límites: 90° para la rotación y 6.0 para la traslación. Por último, esos valores los envié a cada una de las rejas para lograr el funcionamiento esperado.

Bloques de código generado

Window.h

```
GLboolean mover;
```

```
GLboolean getMover() { return mover; }
```

Window.cpp

```
if (key == GLFW_KEY_M && action == GLFW_PRESS)
{
    //theWindow->mover = GL_TRUE;
    theWindow->mover = GL_TRUE;
}

if (key == GLFW_KEY_N && action == GLFW_PRESS)
{
    theWindow->mover = GL_FALSE;
    //theWindow->mover = !theWindow->mover;
}
```

P09-320109766.cpp

```
float rotpuerta;
float rotpuertaOffset;
float despuerta;
float despuertaOffset;
```

```
rotpuerta = 0.0f;
rotpuertaOffset = 1.0f;
despuerta = 0.0f;
despuertaOffset = 0.066f;
```

```

// Movimiento rotacion
if ( mainWindow.getMover() ) {
    if (rotpuerta > -90.0f)
    {
        rotpuerta -= rotpuertaOffset * deltaTime;
    }
}
else {
    if (rotpuerta < 0.0f)
    {
        rotpuerta += rotpuertaOffset * deltaTime;
    }
}

// Movimiento traslacion
if (mainWindow.getMover()) {
    if (despuerta < 6.0f)
    {
        despuerta += despuertaOffset * deltaTime;
    }
}
else {
    if (despuerta > 0.0f)
    {
        despuerta -= despuertaOffset * deltaTime;
    }
}
}

```

```

//Reja 1
model = modelaux;
model = glm::translate(model, glm::vec3(6.0f, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.03f, 0.03f, 0.03f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, -1.0f, 0.0f));
model = glm::rotate(model, rotpuerta * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta.RenderModel();

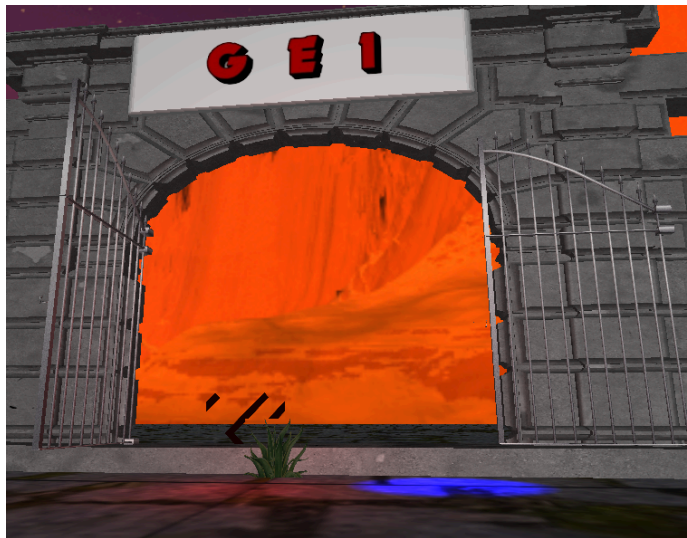
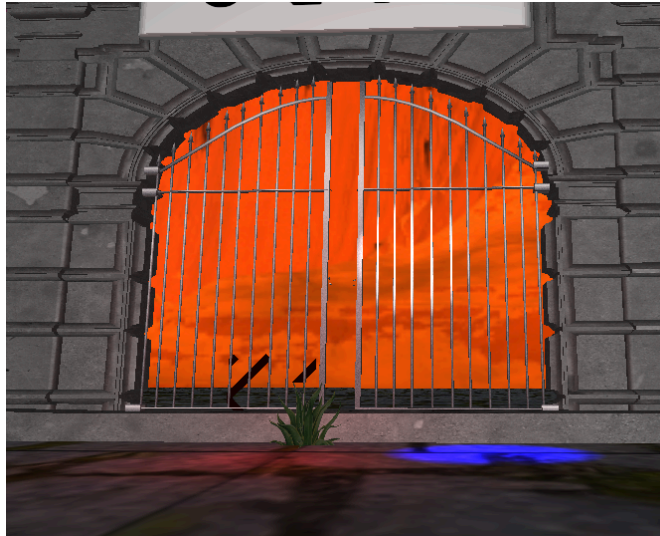
```

```

//Reja 2
model = modelaux;
model = glm::translate(model, glm::vec3(-7-despuerta, 0.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.03f, 0.03f, 0.03f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta.RenderModel();

```


Código en ejecución



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Con esta práctica afortunadamente no tuve inconvenientes.

Conclusión:

Con estos ejercicios aprendí a crear dos animaciones diferentes con las rejillas y también logré hacer un letrero LED usando una textura en movimiento. Además, reforcé mis conocimientos en el uso de Blender y GIMP. Fue una práctica muy entretenida, y me da gusto saber que ya tengo casi todo lo necesario para poder terminar mi proyecto final.

Bibliografía :

OpenGL Architecture Review Board, Shreiner, D., Woo, M., Neider, J., & Davis, T. (2010). OpenGL Programming Guide: The Official Guide to Learning OpenGL (7th ed.). Addison-Wesley.

OpenGL. (s. f.). OpenGL Documentation. Khronos Group. Recuperado de <https://www.khronos.org/opengl/>

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.